

Homework 1 Solutions - Spring 2026 - STAT 8330

Paul Schliekelman

Problem 2: Heteroskedasticity

Introduction

One of the fundamental assumptions of linear regression is that of *homoscedasticity*, that is that the residual variance is equal across the range of the data. In this problem, we will use simulations to examine how the performance of standard linear regression is impacted when this assumption is violated.

We will look at how heteroskedasticity impacts inference of the slope. We will look at three questions:

1. In a hypothesis test about the value of the slope of a line, is the false positive rate impacted by heteroskedasticity?
2. In a hypothesis test about the value of the slope of a line, is the power impacted by heteroskedasticity?
3. Is the accuracy of the estimated slope affected by heteroskedasticity?

Simulation Procedure

We will examine these questions via simulations. In the simulations, we will generate linear regression data in which the variance of the residuals varies in different ways with respect to the predictor X . We will conduct statistical inference about the slope of the regression line and determine whether the inference is impacted by the variation of the variance.

The simulation program is of form

loop over simulation replicates

```
{ 1. Generate  $x$  data  
2. generate  $y$  data from linear model  $y = a + bx + \epsilon$  with  $\epsilon \sim normal(0, \sigma^2(x))$   
3. fit the  $y$  data to the  $x$  data using the lm function  
4. In each replicate, save the fitted slope coefficient and the p-value for the test of whether the slope is different than zero.  
}
```

output:

rejection probability = (number of p-values < 0.05) / (number of replicates)

simulation CI for the estimate of the slope is the middle 95% of the slope estimates

We will use the distribution of the slope estimates to determine whether the accuracy of the slope estimates are affected by heteroskedasticity.

Depending on whether the true slope is zero or not, the rejection probability gives us either the false positive rate or the power. We will compare these across different scenarios in order to determine whether they are impacted by heteroskedasticity.

Simulation code Here is the simulation program:

```
#function to simulate linear regression
#function input:
#n: sample size
#a,b: intercept and slope for true relationship between y and x
#sd0: the standard deviation of residuals. Can be a vector of values if SD changes with x
#nrep: number of simulation replicates

#function output:
#list with 1) probability that the null hypothesis of slope zero is rejected, given inputs.
# 2) mean estimate of slope across replicates
#3) the middle 95% of slope estimates
#The function plots the last data set for purposes of visualization.

linregsim<-function(n,a,b,sd0,nrep,makeplot=T)
{
  x=seq(from=0,to=10,length.out=n) #generate x data
  coef=vector(length=nrep) #vector to hold slope estimates
  pval=vector(length=nrep) #vector to hold p-values

  for(r in 1:nrep) #loop over sim replicates
  {
    y=a+b*x+rnorm(n,mean=0,sd=sd0) #generate y data
    linmod=lm(y~x) #fit the model
    coef[r]=linmod$coefficients[2] #save the slope
    pval[r]=summary(linmod)$coefficients[2,4] #save the p-value.
  }

  if(makeplot==T)
  {plot(x,y)
   abline(a=a,b=b)}

  CI=quantile(coef,probs=c(0.025,0.975)) #middle 95% of slope estimates

  out=list(reject_prob=sum(pval<0.05)/nrep, mean_bhat=mean(coef),CI=CI )
  return(out)
}
```

Simulation scenarios We will consider four different scenarios for how the variance varies with X :

1. Constant variance.
2. Variance increasing linearly with X .
3. Variance decreasing linearly with X .
4. Variance has a parabolic shape with respect to X .

Results

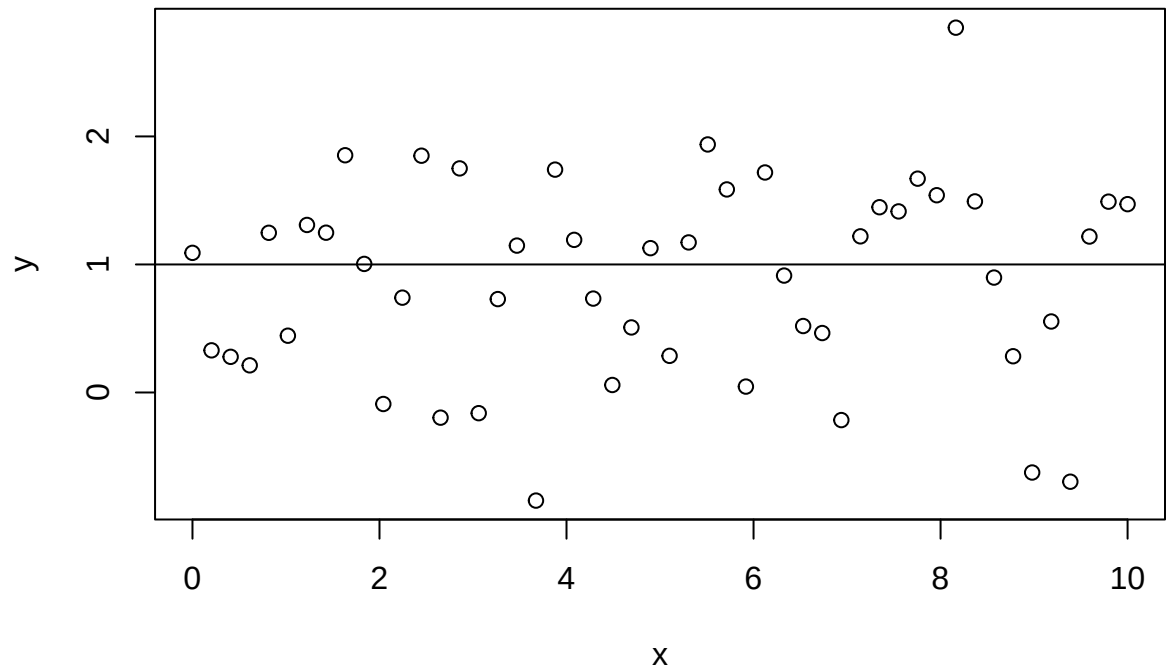
For each variance scenario, we will conduct simulations with a slope of zero, 0.05, and 0.1. In the case with slope 0.05 there is a low power to detect that the slope is different than zero with the standard deviation of 0.5 and sample size 50. In the case of slope 0.1, the power is relatively high.

We organize the results in terms of false positive rate, power, and accuracy of estimates.

False Postive Rate - slope 0

In order to determine the effect of heteroskedasticity on false positive rates, we will set the slope to zero and run the function to conduct simulations of the rejection rate for the four different scenarios for variance

```
n0=50  
linregsim(n=n0,a=1,b=0,sd0=1,nrep=10000)$reject_prob
```



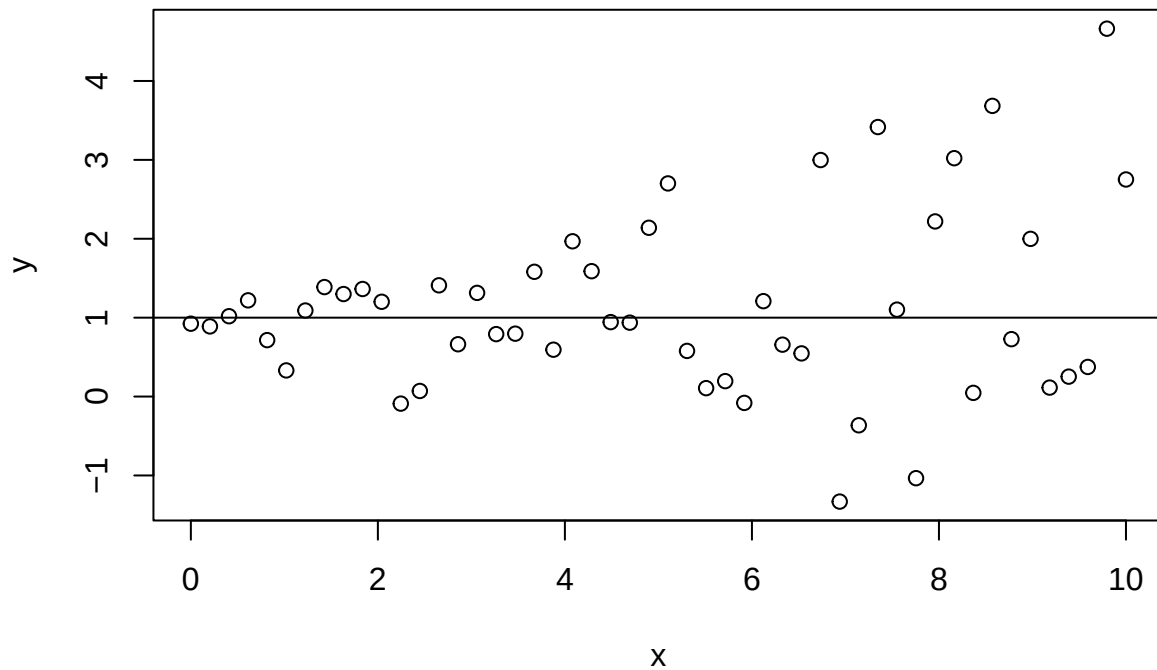
Equal variance

```
## [1] 0.0507
```

When the true slope is zero, then the probability of rejecting the null is the probability of making a false positive. As expected, the false positive rate is very close to 0.05.

Linear Increase in Standard Deviation In this scenario, the standard deviation steadily increases from 0.1 to 2 as x increases.

```
linregsim(n=n0,a=1,b=0,sd0=seq(from=0.1,to=2,length.out=n0),nrep=10000)$reject_prob
```

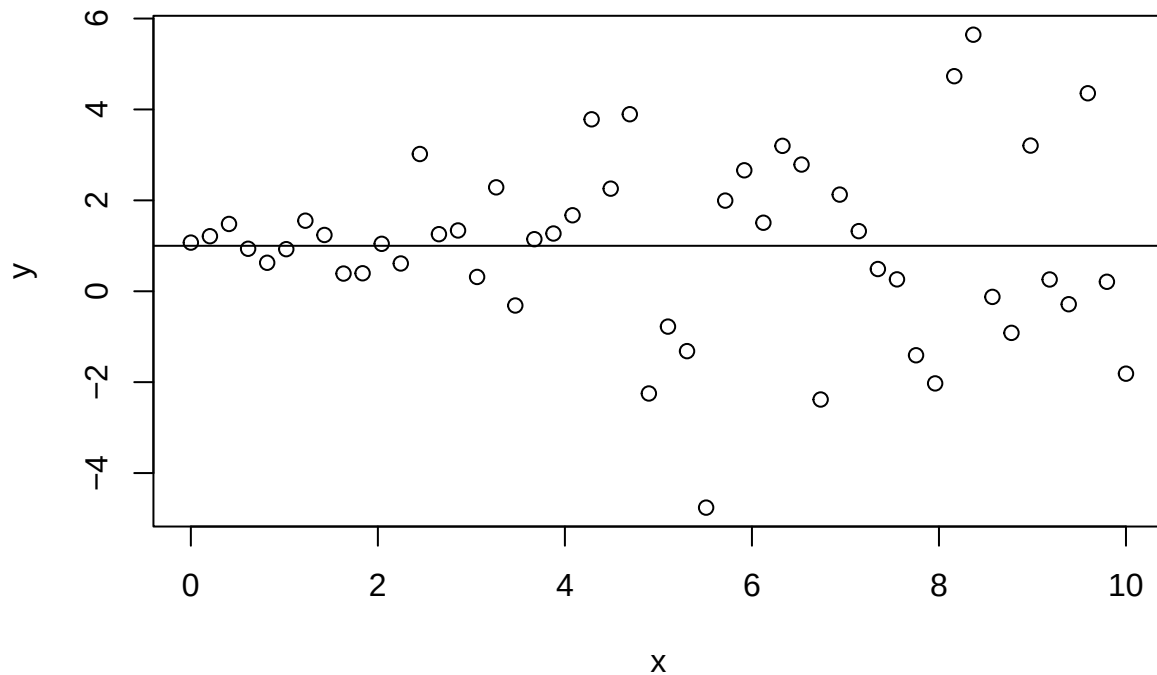


```
## [1] 0.0688
```

When there is a linear increase in variance, we see that the false positive rate is about 7%. So there is an inflation in false positive rate.

Next, we double the size of the increase in variance:

```
linregsim(n=n0,a=1,b=0,sd0=seq(from=0.1,to=4,length.out=n0),nrep=10000)$reject_prob
```

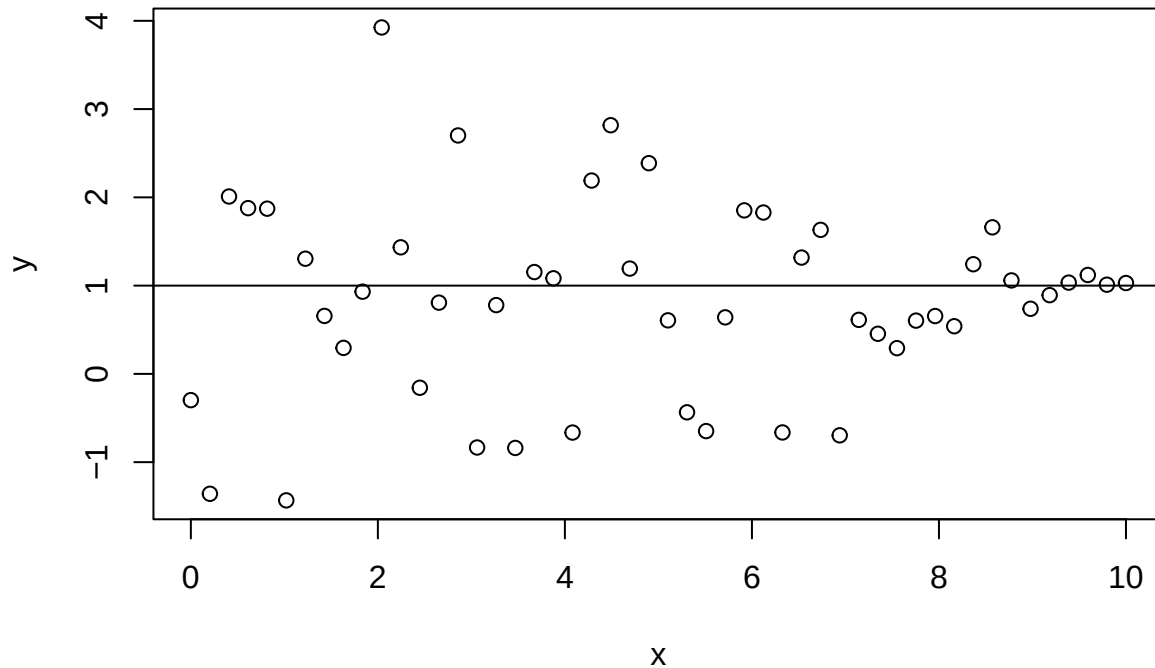


```
## [1] 0.0772
```

This does not substantially increase the false positive rate compared to the case with a slower rate of increase.

Linear Decrease in standard deviation In this scenario, the standard deviation steadily decreases from 2 to 0.1 as x increases.

```
linregsim(n=n0,a=1,b=0,sd0=seq(from=2,to=0.1,length.out=n0),nrep=10000)$reject_prob
```

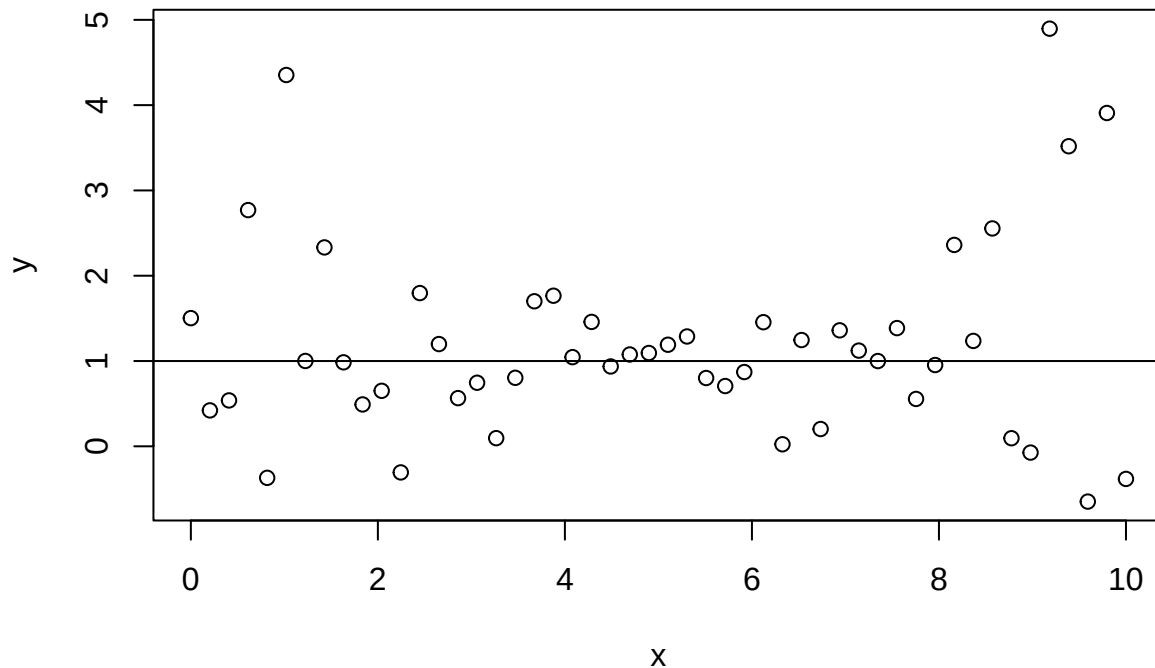


```
## [1] 0.0719
```

With a decrease in variance of the same magnitude, we see that the false positive rate is again inflated to about 7%.

Parabolic Standard Deviation In this scenario, the standard deviation steadily decreases from 2 to 0.1 and then back up to 2 as x increases.

```
linregsim(n=n0,a=1,b=0,sd0=c(seq(from=2,to=0.1,length.out=n0/2),seq(from=0.1,to=2,length.out=n0/2)),nrep=10000)$reject_prob
```



```
## [1] 0.1374
```

With the parabolic variation in standard deviation, the false positive rate doubles to over 14%. This is nearly three times the target false positive rate.

Overall, we see that the false positive rate is inflated, potentially by a lot when the assumption of homoscedasticity is violated.

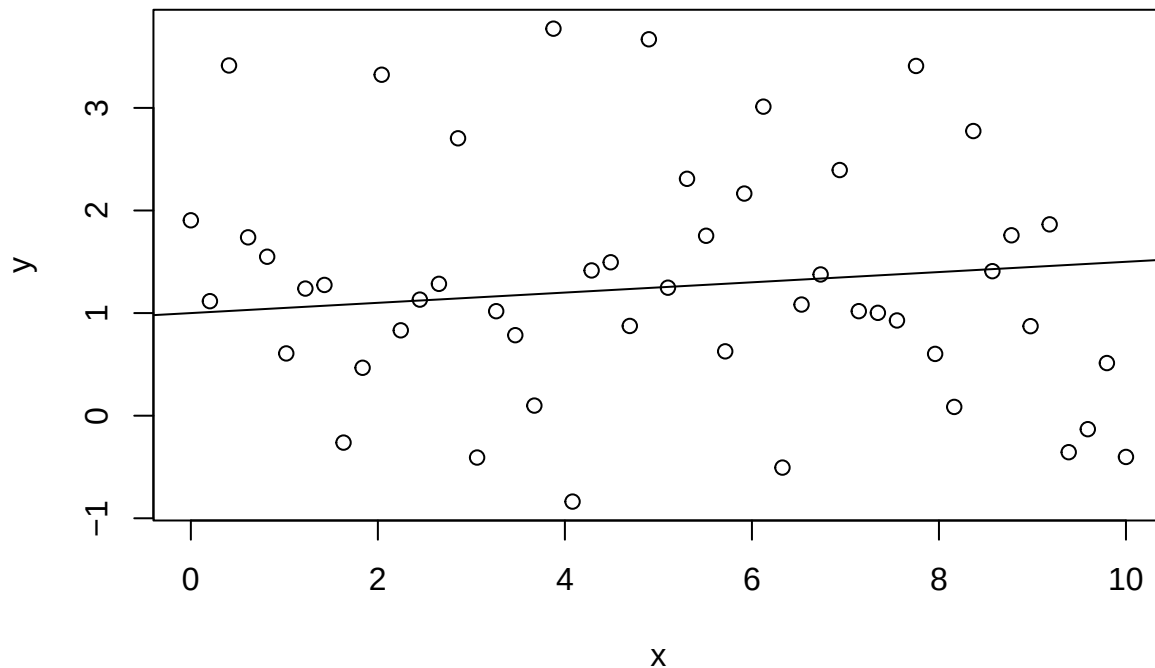
Power

Next, we will examine the power by simulating the rejection probability when the slope is different than zero. We will look at cases of slope=0.05 and slope=0.1. The first case will give fairly low power - around 15%. The second case will give higher power - around 50%.

When variance is changing with X , there are two potential effects on power. First, there is the direct effect of variance on power - if we increase the residual variance, then power will decrease. Second, the hypothesis testing procedure may make mistakes caused by varying variance that impacts power.

Equal variance Start with the case with of equal fixed SD. The SD of 1.05 is chosen to be the same as the average SD for the cases of the linear increase below.

```
(pow1=linregsim(n=n0,a=1,b=0.05,sd0=1.05,nrep=10000)$reject_prob)
```



```
## [1] 0.1629
```

With constant variance, true slope of 0.05 and sample size 50, the power to detect that the slope is different than zero is about 18%.

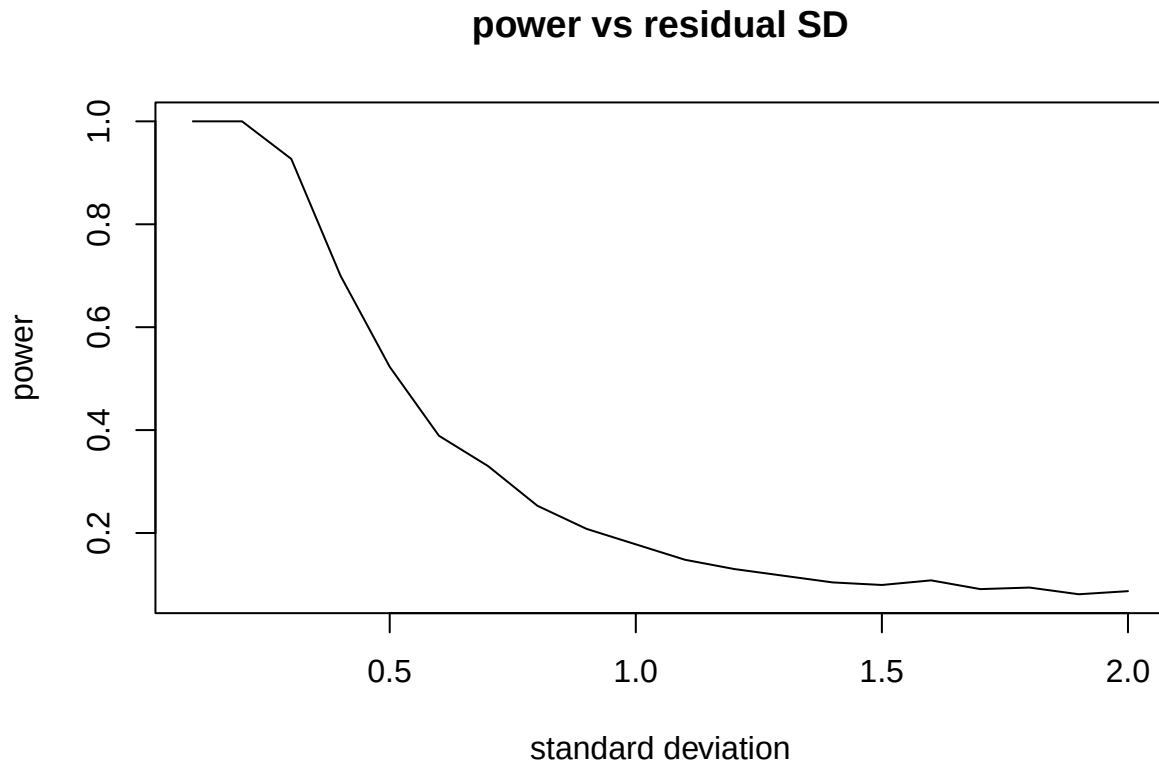
Linear Increase in Standard Deviation Next, we will have a linear increase in SD.

In the first plot below, simulations are conducted for a range of values of standard deviation, but with the standard deviation fixed within a study.

```
n0=50

sdseq=seq(from=0.1,to=2,by=0.1)
powvec=vector(length=length(sdseq))
i=1
for(sd1 in sdseq)
{
  powvec[i]=linregsim(n=n0,a=1,b=0.05,sd0=sd1,nrep=1000, makeplot=FALSE)$reject_prob
  i=i+1
}

plot(sdseq,powvec,type="l", xlab="standard deviation",ylab="power",main="power vs residual SD")
```



We see that the power follows a typical reverse S shape: decreasing slowly from 1 when SD is small, then a linear decrease in the range of about 0.5 to 1, and then flattening out and approaching zero slowly as the SD increases to 2.

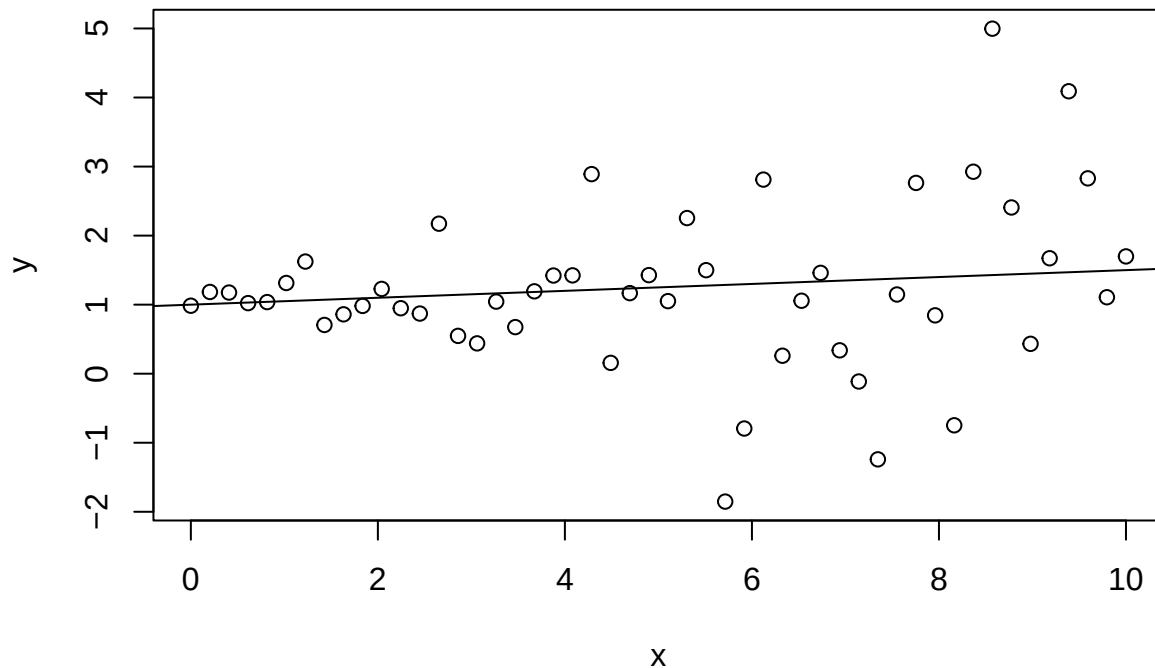
The mean and median power are 0.33 and 0.17, respectively:

```
summary(powvec)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.0810  0.1027  0.1630  0.3283  0.4225  1.0000
```

Next, we will return to the scenario where the SD is varying with X within samples:

```
linregsim(n=n0,a=1,b=0.05,sd0=seq(from=0.1,to=2,length.out=n0),nrep=10000)$reject_prob
```



```
## [1] 0.1675
```

The power appears to decrease slightly relative to the constant case. The power is similar to the median power but lower than the mean power for the plot with varying SD above. The difference between these plots is that in the first plot the SD is fixed within studies. Each simulation replicate has constant variance. The power values are each from 1000 simulation replicates of regression studies with constant variance. In the second plot, SD is varying with studies. Each replicate is of one study with SD varying across X. The power is from 10,000 simulation replicates of regression studies with varying SD.

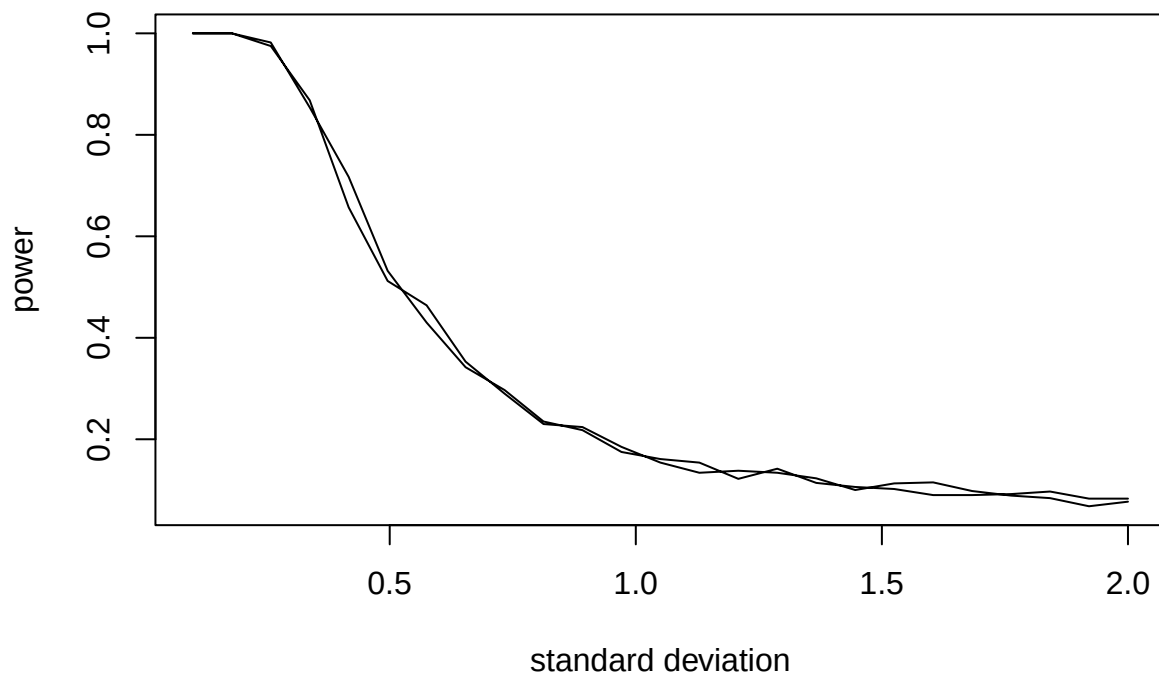
Parabolic Standard Deviation For the case of parabolic variation in SD, we will again conduct one simulation with SD constant within studies and varying across studies and one simulation with SD varying within studies.

```
n0=50

sdseq=c(seq(from=2,to=0.1,length.out=n0/2),seq(from=0.1,to=2,length.out=n0/2))
powvec=vector(length=length(sdseq))
i=1
for(sd1 in sdseq)
{
  powvec[i]=linregsim(n=n0,a=1,b=0.05,sd0=sd1,nrep=1000, makeplot=FALSE)$reject_prob
  i=i+1
}

plot(sdseq,powvec,type="l", xlab="standard deviation",ylab="power",main="power vs residual SD")
```

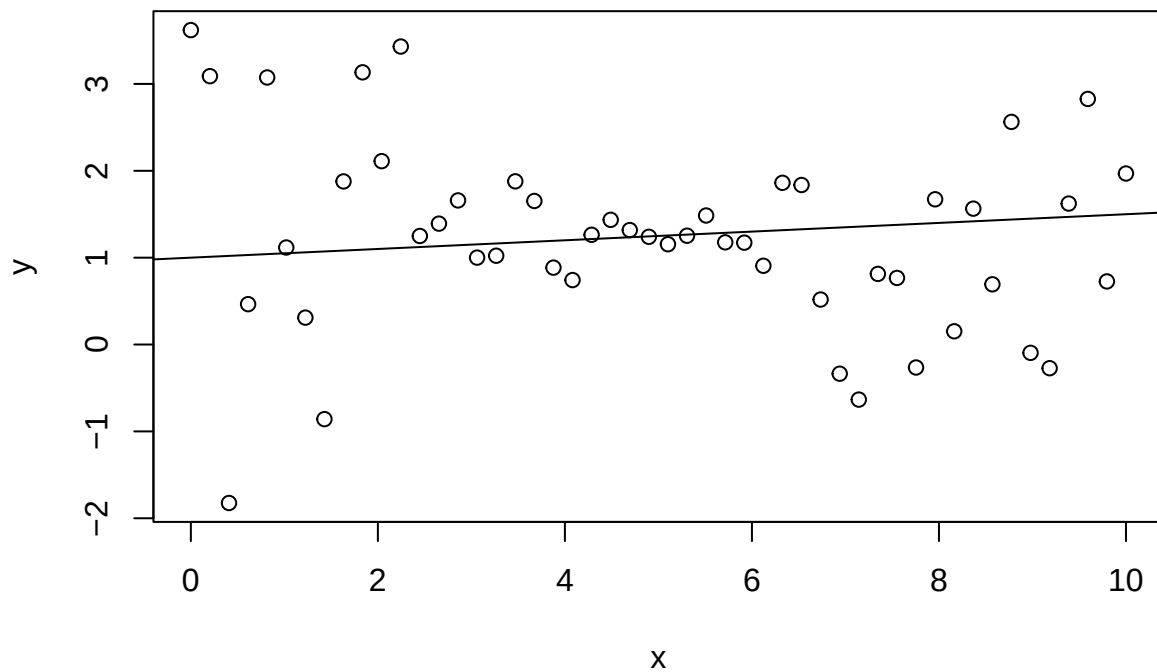
power vs residual SD



```
summary(powvec)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.0680 0.1030 0.1575 0.3281 0.4555 1.0000
```

```
linregsim(n=n0,a=1,b=0.05,sd0=c(seq(from=2,to=0.1,length.out=n0/2),seq(from=0.1,to=2,length.out=n0/2)),r
```

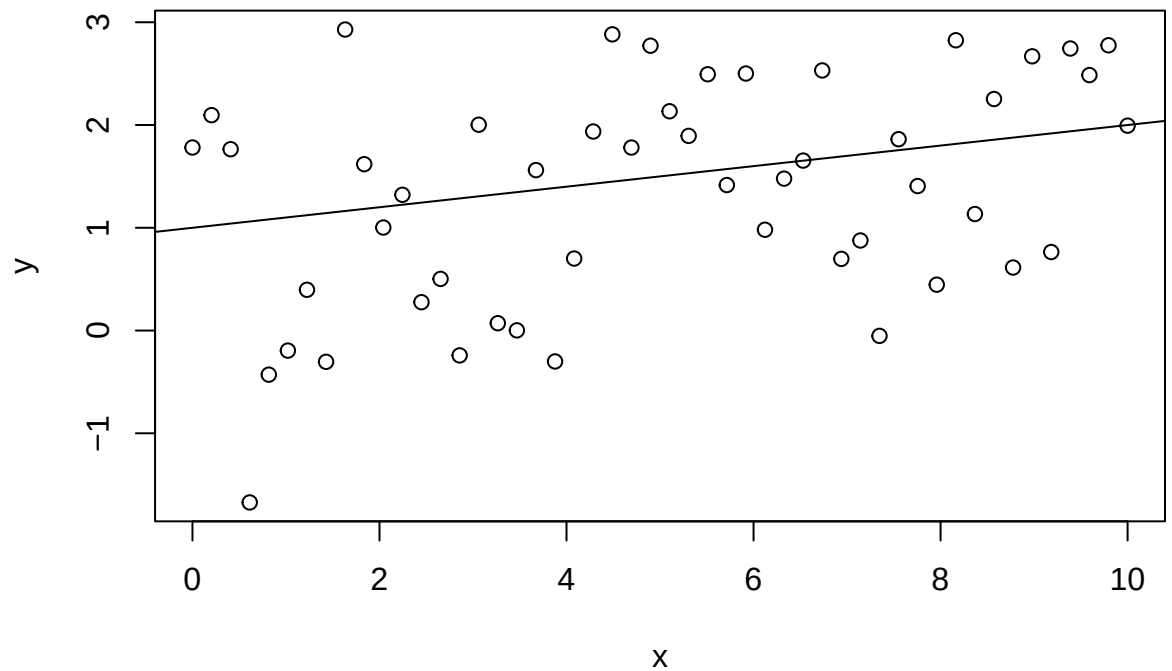


```
## [1] 0.2292
```

With the parabolic variation in standard deviation, the power for the case of heteroskedasticity is slightly higher than the constant variance case and is intermediate between the median and mean of the case with SD varying parabolically between studies.

Slope=0.1 Now, we increase the true slope to 0.1, to give a scenario with higher power.

```
(pow1=linregsim(n=n0,a=1,b=0.1,sd0=1.05,nrep=10000)$reject_prob)
```



Equal variance

```
## [1] 0.4997
```

With constant variance, true slope of 0.1 and sample size 50, the power to detect that the slope is different than zero is about 53%.

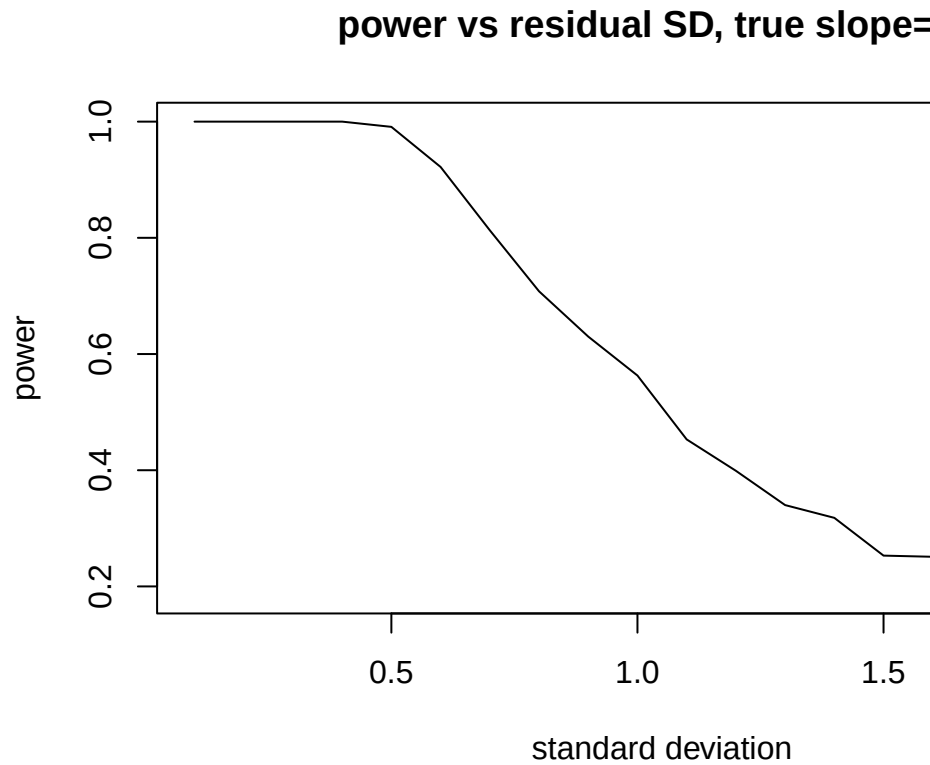
```
n0=50

sdseq=seq(from=0.1,to=2,by=0.1)
powvec1=vector(length=length(sdseq))
i=1
for(sd1 in sdseq)
{
  powvec1[i]=linregsim(n=n0,a=1,b=0.1,sd0=sd1,nrep=1000, makeplot=FALSE)$reject_prob
  i=i+1
}
```

```
}
```

```
par(mfrow=c(1,1))
```

```
plot(sdseq,powvec1,type="l", xlab="standard deviation",ylab="power",main="power vs residual SD, true slope=
```



Linear Increase in Standard Deviation

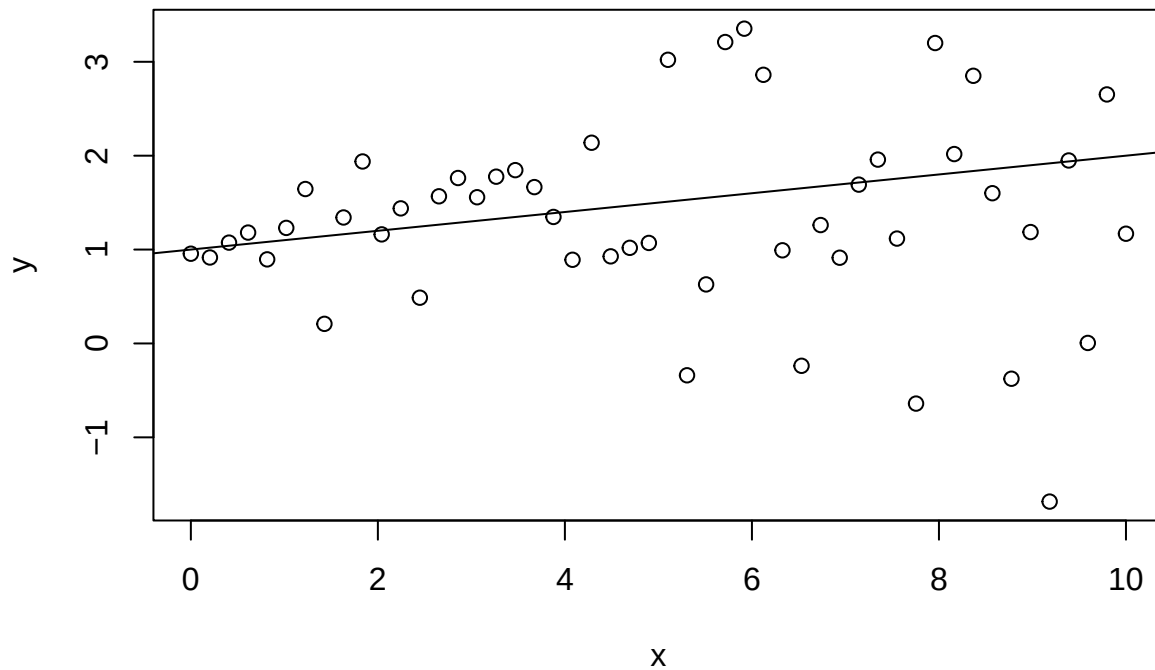
```
summary(powvec1)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.1860  0.2525  0.5080  0.5741  0.9393  1.0000
```

Interestingly, for the higher power scenario, there is not much difference in the mean and median power. The median power is nearly identical to that for the constant variance case and the mean is higher by about 5%.

Now we have SD varying within samples:

```
linregsim(n=n0,a=1,b=0.1,sd0=seq(from=0.1,to=2,length.out=n0),nrep=10000)$reject_prob
```



```
## [1] 0.4168
```

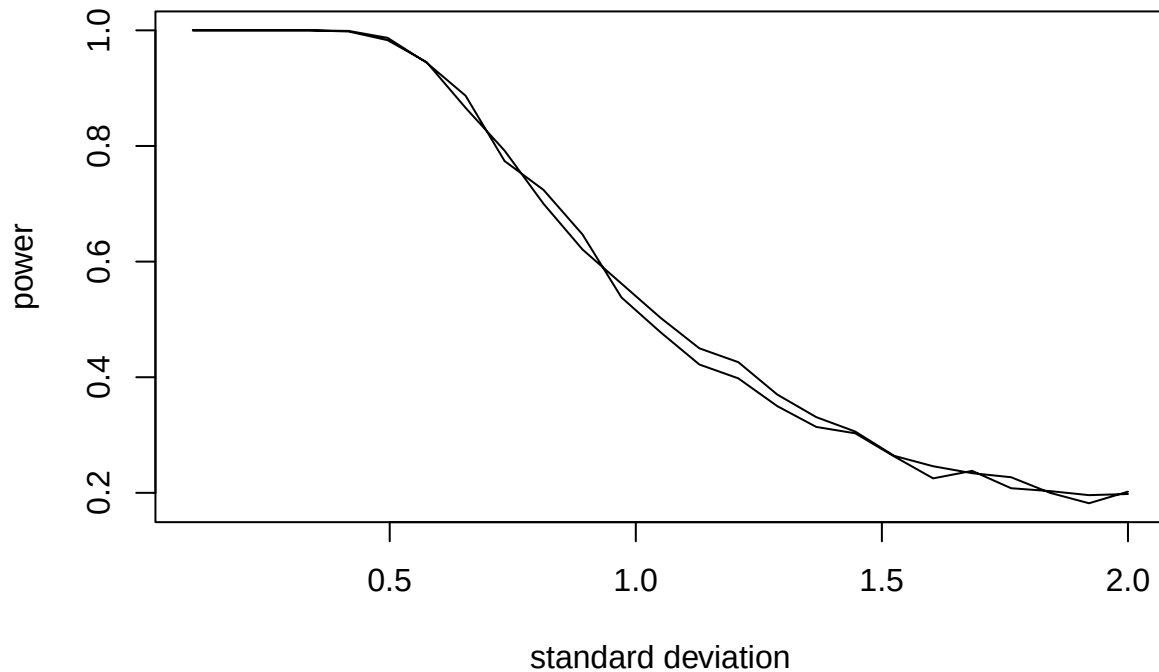
Parabolic Standard Deviation Somewhat unexpectedly, we see the power in the parabolic case to decrease with similar magnitudes as the linear cases. I call this unexpected because there were substantial differences in false positive rate and in power with slope=0.05 between the linear and parabolic cases.

```
n0=50

sdseq2=c(seq(from=2,to=0.1,length.out=n0/2),seq(from=0.1,to=2,length.out=n0/2))
powvec2=vector(length=length(sdseq2))
i=1
for(sd1 in sdseq2)
{
  powvec2[i]=linregsim(n=n0,a=1,b=0.1,sd0=sd1,nrep=1000,makeplot=FALSE)$reject_prob
  i=i+1
}

plot(sdseq2,powvec2,type="l", xlab="standard deviation",ylab="power",main="power vs residual SD")
```

power vs residual SD



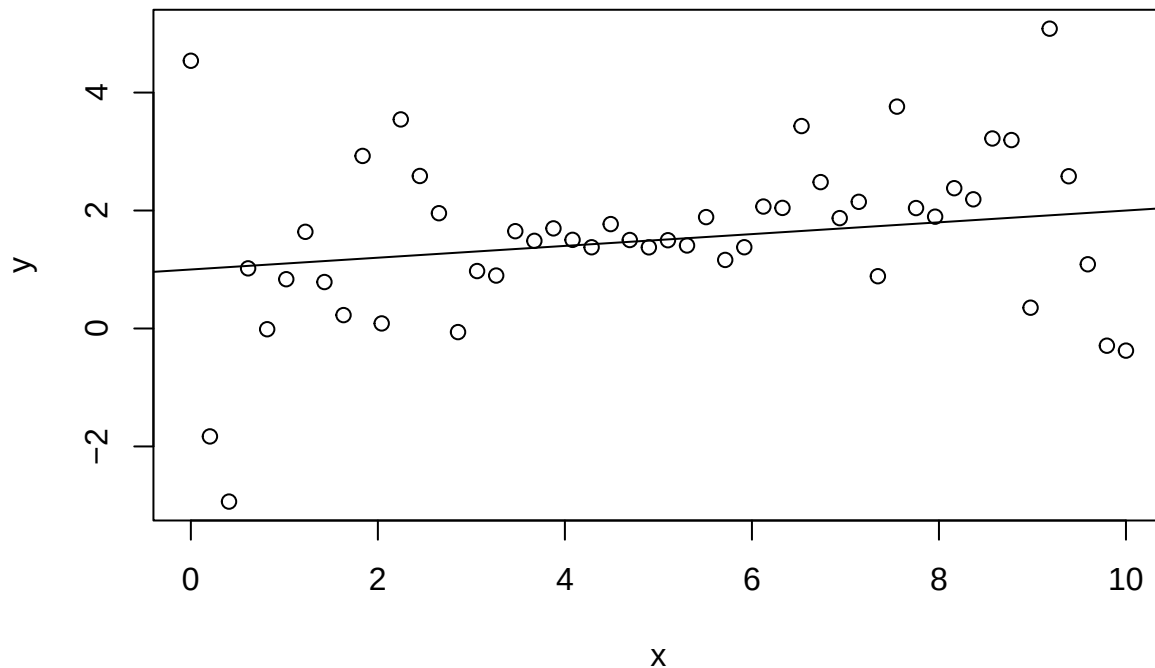
```
summary(powvec2)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.1820  0.2632  0.4905  0.5741  0.9447  1.0000
```

The power is very similar between the case with parabolic variation in SD and the one with linear decrease when SD is constant within study.

Again, when SD is varying between studies, the power decreases by about 10% relative to the constant variance case.

```
linregsim(n=n0,a=1,b=0.1,sd0=c(seq(from=2,to=0.1,length.out=n0/2),seq(from=0.1,to=2,length.out=n0/2)),n
```



```
## [1] 0.4257
```

Accuracy of slope estimate

Finally, we will examine the accuracy of the slope estimates. We will compare the mean estimate of the slope and the middle 95% of slope estimates for each of the scenarios for variation in SD that we have considered.

The output `mean_bhat` is the mean estimate of the slope across replicates. The true slope is 0.1 for all simulations. The mean estimate is very close to 0.1 for all of the simulations. This tells us that the estimate of the slope is not biased by heteroskedasticity.

In the case of constant variance ranging, the middle 95% of slope estimates ranges from just above zero to about 0.19. With a linear increase in SD from 0.1 to 2, the middle 95% of slope estimates ranges from about -0.02 to 0.22 and this interval increases to -0.14 to 0.33 when the SD increases linearly from 0.1 to 4. The variation in the middle 95% of slope estimates is similar for a linear decrease in SD. When SD varies parabolically from 0.1 up to 2 and then back down again, the middle 95% of slope estimates is about -0.05 to 0.25.

Overall, we see that estimates of slope are unbiased under heteroskedasticity, but can be more variable.

```
n0=50
linregsim(n=n0,a=1,b=0.1,sd0=1,nrep=10000, makeplot = FALSE)
```

```
## $reject_prob
## [1] 0.5383
##
## $mean_bhat
## [1] 0.1004297
##
## $CI
##      2.5%      97.5%
## 0.006262265 0.193510834
```

```
linregsim(n=n0,a=1,b=0.1,sd0=seq(from=0.1,to=2,length.out=n0),nrep=10000, makeplot = FALSE)
```

```
## $reject_prob
## [1] 0.4164
##
## $mean_bhat
## [1] 0.09986689
##
## $CI
##      2.5%      97.5%
## -0.02212077  0.22535563
```

```
linregsim(n=n0,a=1,b=0.1,sd0=seq(from=0.1,to=4,length.out=n0),nrep=10000, makeplot = FALSE)
```

```
## $reject_prob
## [1] 0.1673
##
## $mean_bhat
## [1] 0.1008737
##
## $CI
##      2.5%      97.5%
## -0.1416454  0.3410813
```

```
linregsim(n=n0,a=1,b=0.1,sd0=seq(from=2,to=0.1,length.out=n0),nrep=10000, makeplot = FALSE)
```

```
## $reject_prob
## [1] 0.4232
##
## $mean_bhat
## [1] 0.1008862
##
## $CI
##      2.5%      97.5%
## -0.01845197  0.22178578
```

```
linregsim(n=n0,a=1,b=0.1,sd0=c(seq(from=2,to=0.1,length.out=n0/2),seq(from=0.1,to=2,length.out=n0/2)),n
```

```
## $reject_prob
## [1] 0.4304
##
## $mean_bhat
## [1] 0.09899419
##
## $CI
##      2.5%      97.5%
## -0.0507128  0.2499718
```

Problem 3: Confidence Intervals

In order to study the coverage properties of the confidence intervals, we will conduct a simulation study. I will assume that the data sets consist of n values independently generated from a $normal(\mu, \sigma)$ distribution. Confidence intervals will be calculated in the standard way:

$$CI = \bar{X} \pm t \frac{s}{\sqrt{(n)}}$$

or

$$CI = \bar{X} \pm z \frac{s}{\sqrt{(n)}}$$

Where $\bar{X}$ is the sample mean, s is the sample standard deviation, n is the sample size, and t and Z are the appropriate values from the t- Z-distributions. Our question of interest is whether the calculated confidence intervals achieve their target coverage. That is, for example, whether the 95% confidence interval contains the true parameter value 95% of the time.

The basic structure of the simulation is

loop r over replicates

- {
- 1. **generate data set**
- 2. **calculate CI for data set**
- 3. **check whether CI includes true parameter value**
- }

$$\text{CI coverage} = \frac{\text{number of replicate with CIs covering true value}}{\text{number of simulation replicates}}$$

I will make a function to do the simulation. I use vectorized calculations. The structure of my program is different, but it does the same thing as the above simulation outline:

1. Use the `matrix` and `rnorm` functions to generate a matrix with `R` rows and `N` columns, where each row is the data set for one replicate.
2. Use the `apply` function to calculate the sample means and standard deviations for each row and store in a vector.
3. Use the vectors of means and standard deviations to calculate the lower and upper bounds for the confidence intervals for each replicate. I calculate two versions of the CIs: one using the t-statistic and one using the Z-statistic.
4. Calculate the coverage for each CI using boolean operations to count the number of replicates where the CI contains the true mean `mu0`.
5. Output the coverage for both methods of calculating the CI.

Here is the function

```
CI_sim<-function(p=0.95,mu0=0,sd0=1,N=30,R=1000)
{
  #generate a matrix with each row holding one simulated data set
  dat1<-matrix(rnorm(N*R,mean=mu0,sd=sd0),nrow=R)

  #calculate the sample mean and st dev for each simulated data set:
  meanvec=apply(dat1,1,mean)
  sdvec=apply(dat1,1,sd)
```

```

#calculate the CI using the t:
CIlower<-meanvec-qt((1-p)/2,df=N-1,lower.tail = F)*sdvec/sqrt(N)
CIupper<-meanvec+qt((1-p)/2,df=N-1,lower.tail = F)*sdvec/sqrt(N)

#calculate the CI using the Z:
CIlowerZ<-meanvec-qnorm((1-p)/2,lower.tail = F)*sdvec/sqrt(N)
CIupperZ<-meanvec+qnorm((1-p)/2,lower.tail = F)*sdvec/sqrt(N)

#Calculate the coverage:
coverage=sum((CIlower<mu0)&(CIupper>mu0))/R
coverageZ=sum((CIlowerZ<mu0)&(CIupperZ>mu0))/R

#make the output data frame:
dat<-data.frame(c("t","Z"),c(coverage,coverageZ))
names(dat)=c("method","coverage")

return(dat)

}

```

Test the function:

```
CI_sim()
```

```
## method coverage
## 1      t      0.954
## 2      Z      0.943
```

Now let's try some different sample sizes:

```
CI_sim(N=100,R=10000)
```

```
## method coverage
## 1      t      0.9505
## 2      Z      0.9478
```

```
CI_sim(N=50,R=10000)
```

```
## method coverage
## 1      t      0.9501
## 2      Z      0.9442
```

```
CI_sim(N=30,R=10000)
```

```
## method coverage
## 1      t      0.9473
## 2      Z      0.9366
```

```
CI_sim(N=20,R=10000)
```

```
##    method coverage
## 1      t    0.9494
## 2      Z    0.9341
```

```
CI_sim(N=10,R=10000)
```

```
##    method coverage
## 1      t    0.9488
## 2      Z    0.9154
```

```
CI_sim(N=5,R=10000)
```

```
##    method coverage
## 1      t    0.9459
## 2      Z    0.8701
```

At larger sample sizes (>50), both methods produce the correct coverage. While the method using the t-distribution maintains the correct 95% coverage at all sample sizes, that based on the Z-distribution has reduced coverage as the sample size decreases. At a sample size of 10, the 95% confidence interval only contains the true parameter value about 92% of the time and at a sample size of 5 it only contains the true parameter value about 88% of the time. This is because the use of the Z-distribution assumes that the variance is known exactly. In reality, we are estimating the variance. The t-distribution accounts for the uncertainty resulting from using an estimate of the variance.